

S.I.G.S.M.

Programación Fullstack

EVEdev

ROL	C.I	APELLIDO	NOMBRE	E-MAIL
Coordinador	5742262-3	Chiessa	Emiliano	emiliano603099@gmail.com
Subcoordinador	5745443-2	Gomez	Ezequiel	ezequielg0o0mez23@gmail.com
Integrante 1	5771900-0	Burgueño	Valentino	valentinoburgeno16@gmail.com

Docente: Acosta, Fabián

Fecha de culminación:

02/09/26

SEGUNDA ENTREGA

I.S.B.O. **3°MJ**

Índice

1. Introducción **2**
2. Lenguaje Backend **2**
2.1. ¿Qué es PHP? 2
2.2. ¿Para qué sirve? 2
2.3. ¿En que se destaca? 2
2.4. Ventajas en sistemas institucionales 3
2.5. ¿Porque PHP es adecuado para el backend del sistema del proyecto S.I.G.S.M.? 4
3. Base de Datos **5**
3.1. ¿Qué es MySQL? 5
3.2. Persistencia de datos clínicos y operativos 5
3.3. Integridad y consistencia de los datos 6
3.4. Compatibilidad con el entorno del hospital 6
3.5. Relación con PHP 6
4. Framework frontend **7**
4.1 ¿Qué es Bootstrap? 7
4.2. Ventajas 7
4.3. Interfaces administrativas 7
4.4. Interfaces móviles 8
4.5. Sistema de grillas responsivo 8
5. Herramientas de control de versiones **9**
5.1. ¿Qué es Git? 9
5.2. Ventajas de Git 9
5.3. ¿Qué es Github? 9
5.4. ¿Para qué sirve? 10
5.5. Requisito del proyecto 10
5.6. Buenas prácticas profesionales 11
6. Entorno de desarrollo **12**
6.1. ¿Qué es Visual Studio Code? 13
6.2. Funciones que nos ofrece 13
6.3. Extensiones 14
7. Necesidades concretas del sistema **14**
7.1. Acceso desde móviles (QR) 14
7.2. Gestión administrativa 15
7.3. Persistencia de datos de traslados 15
8. Referencias **17**



1. Introducción

En este documento se justificará la selección de tecnologías para el desarrollo de los 2 módulos del proyecto S.I.G.S.M. Se realizará un análisis explicando los motivos de su utilización y las ventajas que presentan, evaluando su aporte directo para el proyecto.

2. Lenguaje Backend

2.1. ¿Qué es PHP?

PHP (oficialmente, este sigle es un acrónimo recursivo para PHP Hypertext Preprocessor) es un lenguaje de scripts generalista y Open Source, creado para el desarrollo de aplicaciones web. Se puede integrar fácilmente al HTML.

2.2. ¿Para qué sirve?

Se ejecuta en el servidor y se integra fácilmente con HTML para crear sitios web, gestionar bases de datos (MySQL) y manejar sesiones de usuario, siendo la base de CMS populares como WordPress.

2.3. ¿En que se destaca?

Lo que distingue a PHP de los lenguajes de script como JavaScript, es que el código se ejecuta en el servidor, generando así el HTML, que será luego enviado al cliente. El cliente solo recibe el resultado del script, sin ningún medio de acceso al código que produjo dicho resultado.

La gran ventaja de PHP es que es extremadamente simple para los principiantes, pero ofrece funcionalidades avanzadas para los expertos.

2.4. Ventajas en sistemas institucionales

PHP es una opción sólida para sistemas institucionales (universidades, entidades gubernamentales, grandes empresas) debido a su madurez, estabilidad y gran compatibilidad técnica. Sus principales ventajas son:

- **Bajo Costo y Eficiencia**: PHP es de código abierto y gratuito, lo que reduce costos de licenciamiento. Su ejecución rápida y eficiente lo hace ideal para aplicaciones empresariales que requieren alto rendimiento.
- **Alta Compatibilidad y Flexibilidad**: Es compatible con la mayoría de los sistemas operativos (Linux, Windows, Unix) y se integra fácilmente con diversas bases de datos (MySQL, PostgreSQL, Oracle).
- **Escalabilidad y Seguridad**: Los frameworks modernos de PHP permiten crear aplicaciones modulares y escalables, capaces de manejar grandes volúmenes de datos y usuarios simultáneos. Además, ofrece herramientas sólidas para la seguridad si se configura adecuadamente.
- **Curva de Aprendizaje y Comunidad**: Posee una sintaxis sencilla y una de las comunidades más grandes del mundo, lo que facilita encontrar soporte, documentación y desarrolladores cualificados.
- **Ideal para Aplicaciones Dinámicas**: Es excelente para procesar formularios complejos, interactuar con bases de datos y generar contenido dinámico en tiempo real, funciones críticas en sistemas de gestión institucional.

2.5. ¿Porque PHP es adecuado para el backend del sistema del proyecto S.I.G.S.M.?

1. PHP fue diseñado originalmente para sistemas tipo Unix, por lo que su compatibilidad y rendimiento con servidores GNU/Linux son nativos y óptimos. Es la base de la famosa pila de desarrollo LAMP (Linux, Apache, MySQL, PHP) y LEMP (Linux, Nginx, MySQL, PHP).
   Este lenguaje se adapta perfectamente a los servidores que el Departamento Técnico de Informática del Hospital de Clínicas actualmente posee, siendo una opción sólida para implementar al sistema.

2. PHP ha superado las tres décadas como pilar de internet. Lejos de quedar obsoleto, ha evolucionado hacia un lenguaje maduro, tipado y de alto rendimiento. Hoy impulsa aproximadamente el 71% de los sitios web con backend conocido, siendo el motor indiscutible de plataformas empresariales y de gestión de contenido masivas, incluyendo gigantes como WordPress, Facebook y Wikipedia.

3. PHP ofrece integración nativa con MySQL mediante dos extensiones principales: PDO (PHP Data Objects) y MySQLi. Ambas permiten realizar consultas, gestionar datos y proteger tu aplicación contra inyecciones SQL.

4. PHP ofrece un ecosistema sumamente robusto para el desarrollo web. Su principal fortaleza radica en una documentación oficial exhaustiva y un soporte comunitario masivo, ideal tanto para principiantes como para desarrolladores experimentados.

5. PHP es altamente capaz de integrarse con sistemas centralizados utilizando módulos independientes. Esta arquitectura descentralizada permite que componentes autónomos se comuniquen con un núcleo central o con servicios externos, facilitando el mantenimiento y la escalabilidad.

3. Base de Datos

El grupo EVEdev se decidió por usar MySQL como el sistema de gestión de base de datos.

3.1. ¿Qué es MySQL?

MySQL es un sistema de gestión de bases de datos de código abierto desarrollado por Oracle. Su reputación en internet lo identifica como una base de datos relacional que permite almacenar, organizar y recuperar datos de manera eficiente. MySQL es utilizado por una amplia variedad de organizaciones y aplicaciones en todo el mundo.

3.2. Persistencia de datos clínicos y operativos

MySQL garantiza que la información de pacientes, historias clínicas y transacciones diarias se almacene de forma segura y recuperable. Esto se logra mediante el motor de almacenamiento InnoDB (transaccional y garante de las propiedades ACID) y la configuración de registros de transacciones para evitar pérdidas ante fallos.

3.3. Integridad y consistencia de los datos

La integridad y consistencia de los datos en MySQL aseguran que estos sean precisos, fiables y válidos a lo largo del tiempo. Para lograrlo, MySQL utiliza restricciones (como PRIMARY KEY y FOREIGN KEY), reglas de validación y el sistema de transacciones ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad).

3.4. Compatibilidad con el entorno del hospital

MySQL es totalmente compatible y nativo para la infraestructura Linux. Linux es el entorno de producción más común para desplegar MySQL, ofreciendo el mejor rendimiento, estabilidad y opciones de configuración.

MariaDB, el motor de base de datos estándar del hospital, nació como una bifurcación (fork) de MySQL y mantiene una alta compatibilidad con este. En la mayoría de los casos, los protocolos, conectores, puertos y sintaxis básica son idénticos, permitiendo que las aplicaciones utilicen ambos sistemas indistintamente.

Todo esto hace que MySQL sea la mejor opción a utilizar para gestionar las bases de datos del hospital, al ser altamente compatible con la infraestructura que posee la base de datos del hospital, permitiendo una implementación mucho más eficiente y rápida.

3.5. Relación con PHP

La relación entre PHP y MySQL es una de las combinaciones más clásicas y potentes en el desarrollo web. Juntos forman la base de aplicaciones dinámicas, donde PHP actúa como el cerebro (lenguaje de servidor) encargado de procesar la lógica, y MySQL funciona como el almacén (base de datos) que guarda y organiza la información.



4. Framework frontend

4.1 ¿Qué es Bootstrap?

Bootstrap es un framework de código abierto para desarrollo web frontend. Creado originalmente por Twitter, sirve para diseñar sitios y aplicaciones adaptables a cualquier dispositivo (responsive) de forma rápida, sin tener que escribir todo el código HTML, CSS y JavaScript desde cero.

4.2. Ventajas

Las ventajas que ofrece este servicio son las siguientes:

- **Ahorro de tiempo**: Proporciona componentes predefinidos y listos para usar, como botones, menús de navegación, ventanas modales, formularios y tarjetas.
- **Diseño adaptable**: Cuenta con un sistema de cuadrícula (grid) que permite organizar el contenido para que se vea bien tanto en computadoras de escritorio como en celulares o tabletas.
- **Consistencia**: Ayuda a mantener un diseño coherente en todo el sitio web estandarizando los estilos.

4.3. Interfaces administrativas

Para el desarrollo de interfaces administrativas, Bootstrap destaca por su extensa biblioteca de componentes reutilizables pre-diseñados (como tablas de datos, formularios estructurados, tarjetas de métricas, modales y alertas) que optimizan los tiempos de programación. Debido a que los paneles internos priorizan la funcionalidad y la gestión de datos masivos sobre la estética, este framework permite a los desarrolladores maquetar sistemas de control y operaciones CRUD (Crear, Leer, Actualizar y Borrar) de forma ágil y estandarizada. Al no tener la necesidad de escribir hojas de estilo complejas desde cero, los equipos técnicos pueden centrar sus esfuerzos en la lógica del negocio, el procesamiento de datos y la integración de APIs, garantizando un panel intuitivo, limpio y fácil de mantener a largo plazo.

4.4. Interfaces móviles

En el ámbito de los dispositivos móviles, Bootstrap es una herramienta clave gracias a su enfoque nativo mobile-first y su potente sistema de cuadrícula (grid system) flexible de 12 columnas. El framework está diseñado estructuralmente para optimizar el código empezando por las pantallas más pequeñas y escalando de forma fluida hacia monitores de escritorio mediante puntos de ruptura (breakpoints) automatizados. Esto asegura que elementos complejos como menús de navegación, imágenes y formularios interactivos se adapten por completo a la ergonomía táctil y a las resoluciones reducidas de los teléfonos inteligentes sin romper el diseño. Al unificar el desarrollo bajo una única base de código responsiva, se elimina la necesidad de crear aplicaciones separadas, garantizando una experiencia de usuario consistente y de alto rendimiento en cualquier dispositivo móvil.



4.5. Sistema de grillas responsivo

El sistema de grillas (grid system) de Bootstrap es una potente herramienta basada en Flexbox que divide el ancho de la pantalla en una cuadrícula de 12 columnas. Utiliza contenedores, filas y columnas para alinear y organizar el contenido de forma automática en cualquier dispositivo.

5. Herramientas de control de versiones

En el grupo EVEdev se seleccionó Git y Github como herramientas de control de versiones

5.1. ¿Qué es Git?

Git es un sistema de control de versiones distribuido. Sirve para rastrear y administrar los cambios en el código fuente de tus proyectos a lo largo del tiempo, permitiéndote "volver al pasado" si algo se rompe, experimentar sin miedo y colaborar en equipo de forma ordenada.

5.2. Ventajas de Git

Lo que Git permite hacer es:

- **Historial completo**: En lugar de tener carpetas llamadas "proyecto_final", "proyecto_final_v2", Git guarda instantáneas del código (commits). Puedes revisar exactamente quién, cuándo y por qué hizo un cambio.
- **Trabajo en equipo**: Permite que varios programadores trabajen en los mismos archivos al mismo tiempo sin sobrescribir el trabajo de los demás.
- **Pruebas seguras (Branches)**: Puedes crear líneas de desarrollo independientes para probar nuevas funciones o corregir errores sin alterar la versión principal del programa.



5.3. ¿Qué es Github?

GitHub es una plataforma en la nube que funciona como el mayor centro de alojamiento de repositorios Git y colaboración para desarrolladores, permitiendo almacenar código, controlar sus versiones y trabajar en equipo desde cualquier lugar. Se considera la red social de los programadores, facilitando la colaboración en proyectos de software, el código abierto y la gestión de tareas.

5.4. ¿Para qué sirve?

Github da las siguientes funciones:

- **Control de Versiones (Git)**: Permite llevar un registro detallado de los cambios en el código, permitiendo regresar a versiones anteriores ("máquina del tiempo") y evitando perder trabajo.
- **Colaboración en Equipo**: Facilita que múltiples desarrolladores trabajen en el mismo proyecto simultáneamente sin sobrescribir el trabajo de otros.
- **Alojamiento en la Nube**: Sirve para respaldar y compartir repositorios de código de forma pública o privada.
- **Proyectos Open Source**: Es el hogar de millones de proyectos de código abierto donde se puede contribuir, explorar y aprender.
- **Herramientas Extra**: Ofrece funcionalidades como GitHub Actions (automatización), GitHub Issues (seguimiento de tareas y errores) y GitHub Pages (alojamiento de sitios web).

5.5. Requisito del proyecto

El repositorio Github del equipo del proyecto es un requisito fundamental para la primera entrega del mismo, ya que mediante este se debe poder acceder a toda la documentación, prototipos, base de datos, etc.

Cabe recalcar que también es necesario para el trabajo colaborativo de los miembros del equipo, permitiendo que varios trabajen a las vez en distintas ramas.

5.6. Buenas prácticas profesionales

Dominar GitHub a nivel profesional implica organizar el código, automatizar procesos y facilitar el trabajo colaborativo. Las prácticas clave incluyen utilizar estrategias de ramas, realizar confirmaciones descriptivas, gestionar el código mediante revisiones obligatorias y mantener el repositorio documentado y seguro.

1. **Estrategia de Ramificación (Branching)**

   Nunca trabajes directamente en la rama principal (main o master).

   - **main / master**: Debe contener únicamente código estable y listo para producción.
   - **develop**: Rama principal de integración para el trabajo en curso (si usas metodologías robustas).
   - **Ramas de función (feature/nombre-tarea)**: Crea una rama nueva para cada tarea o funcionalidad.
   - **Ramas de error o entorno (bugfix/, hotfix/, testing/)**: Aísla los arreglos y despliegues.

2. **Mensajes de Commits Profesionales**

   Tus confirmaciones (commits) son la bitácora del proyecto. Un formato estandarizado como Conventional Commits es el estándar de la industria:

   - **Usa verbos en imperativo**: Add, Fix, Update.
   - **Formato**: <tipo>(<alcance>): <descripción>
   - **Ejemplos**: feat(auth): agregar validación de token JWT o fix(login): corregir error de redirección.

3. **Solicitudes de Extracción (Pull Requests)**

   En lugar de fusionar código directamente, utiliza siempre Pull Requests (PRs).

   - **Descripción clara**: Utiliza las plantillas de PR integradas en GitHub para detallar el problema resuelto y los pasos para verificarlo.
   - **Revisiones de código (Code Reviews)**: Exige que al menos un compañero revise y apruebe el código antes de fusionarlo.



- **Enlaces de seguimiento**: Vincula el PR directamente al Issue o tarea correspondiente.

4. **Configuración y Mantenimiento del Repositorio**

   Mantén el proyecto ordenado y profesional desde la raíz:

   - Archivo README.md: La carta de presentación. Incluye instrucciones de instalación, uso, tecnologías y contribución.
   - LICENSE y CONTRIBUTING.md: Define cómo otros pueden usar tu código y las reglas para aportar al proyecto.
   - Archivo .gitignore: Imprescindible para evitar subir archivos temporales, dependencias pesadas (ej. node_modules/) o credenciales.

5. **Seguridad y Automatización**

   Aprovecha las herramientas nativas y de CI/CD:

   - CODEOWNERS: Define qué usuarios o equipos son responsables de revisar ciertas partes específicas del código.
   - Protección de ramas: Configura reglas en GitHub para evitar eliminaciones accidentales de la rama main y asegurar que el código pase pruebas automáticas antes de unirse.
   - Gestión de secretos: Nunca subas contraseñas o claves API. Usa variables de entorno o el gestor de secretos de GitHub.

6. Entorno de desarrollo

En el grupo EVEdev se seleccionó Visual Studio Code como entorno de desarrollo.



6.1. ¿Qué es Visual Studio Code?

Visual Studio Code (VS Code) es un editor de código fuente gratuito y de código abierto creado por Microsoft. Sirve para escribir, editar, organizar y depurar código en prácticamente cualquier lenguaje de programación (como JavaScript, Python, C++ y HTML/CSS).

6.2. Funciones que nos ofrece

Las funciones que ofrece Visual Studio Code son las siguientes:

- **Asistencia inteligente (IntelliSense)**: Ofrece resaltado de colores para el código y funciones de autocompletado que predicen lo que estás escribiendo, lo que acelera el proceso y evita errores.
- **Depuración integrada (Debugging)**: Permite ejecutar tu código paso a paso y detenerlo en puntos específicos para inspeccionar posibles fallos o errores en el programa.
- **Control de versiones con Git:** Se conecta de forma nativa con herramientas como Git y GitHub, permitiéndote guardar diferentes versiones de tu proyecto y trabajar en equipo sin salir del editor.
- **Terminal integrada**: Incluye una consola de comandos en la parte inferior de la pantalla para ejecutar scripts e interactuar con tu sistema operativo o servidores.
- **Personalización masiva**: Cuenta con un catálogo (Marketplace) con decenas de miles de extensiones y "plugins". Puedes añadirle soporte para nuevos lenguajes, cambiar la apariencia visual (temas) o instalar asistentes de Inteligencia Artificial (como GitHub Copilot).



6.3. Extensiones

Las extensiones a utilizar serán las siguientes:

- **Live Server**: Esta extensión permite correr un servidor de desarrollo local en donde se puede visualizar la página, este servidor se actualiza cada vez que se realiza un guardado en Visual Studio.
- **Live Preview**: Junto a Live Server, esta extensión permite ver en tiempo real los cambios sin necesidad de guardar, además de poder visualizar el diseño responsivo de la página al poder cambiar el tamaño de la ventana.
- **Prettier**: Establece un formato automático del código para que sea más legible y mantener consistencia en el formato a la hora del trabajo cooperativo.
- **GitLens**: Permite visualizar el historial de versiones del proyecto directamente en el editor.
- **PHP Intelephense**: Ofrece autocompletado rápido, análisis estático, navegación de código y diagnósticos en tiempo real para optimizar el flujo de trabajo de desarrollo en PHP.
- **MySQL**: Conecta con la base de datos y permite visualizarla directamente dentro del IDE.

7. Necesidades concretas del sistema

7.1. Acceso desde móviles (QR)

Esta necesidad requiere una interfaz adaptable, ligera y con capacidad de interactuar con hardware móvil (como la cámara para escanear o pantallas pequeñas para mostrar el QR).

- **Bootstrap (Frontend)**: Es clave aquí, al ser un framework mobile-first, garantiza que la interfaz donde se muestra o se escanea el QR se adapte perfectamente a cualquier pantalla de smartphone de los pacientes o personal de salud, sin deformarse.
- **PHP (Backend)**: Se encarga de la lógica detrás del código QR. PHP procesará la solicitud cuando el QR sea escaneado, validará el token o ID encriptado en el código y servirá la información correspondiente en tiempo real.

7.2. Gestión administrativa

Los módulos de administración hospitalaria o institucional requieren un desarrollo rápido, un panel de control (dashboard) robusto, manejo de sesiones seguras y flujos de trabajo internos eficientes.

- **PHP**: Como lenguaje backend en sistemas institucionales, PHP es ideal para la gestión administrativa debido a su madurez y estabilidad. Permite manejar de forma nativa la autenticación de usuarios (roles de administrador, médicos, administrativos), el procesamiento de formularios complejos y la comunicación segura con la base de datos.
- **Bootstrap**: Facilita la creación rápida de interfaces administrativas limpias y organizadas (tablas de datos, gráficos, formularios de registro, botones de acción) mediante sus componentes prediseñados, ahorrando tiempo en el diseño del backend administrativo.



7.3. Persistencia de datos de traslados

La gestión de traslados (médicos, de pacientes o de insumos) es crítica; requiere que los datos operativos y de bitácora no se pierdan, se registren en orden cronológico y mantengan una relación estricta entre el paciente, el vehículo y el destino.

- **MySQL (Base de Datos)**: Es la tecnología responsable de esta necesidad.
    - **Persistencia**: Asegura que cada cambio de estado en un traslado quede grabado de forma permanente.
    - **Integridad y Consistencia**: Mediante el uso de claves foráneas (Foreign Keys) y transacciones ACID, garantiza que no se pueda registrar un traslado para un paciente que no existe, o que los datos operativos no queden duplicados o corruptos si ocurre un fallo en la red a mitad del proceso.

8. Referencias

Arsys. (s.f.). Guía completa sobre Bootstrap. https://www.arsys.es/blog/guia-completa-sobre-bootstrap

Arsys. (s.f.). Qué es Visual Studio Code y cuáles son sus ventajas. https://www.arsys.es/blog/que-es-visual-studio-code-y-cuales-son-sus-ventajas

BootstrapDash. (s.f.). What is Bootstrap framework: Admin dashboards. https://www.bootstrapdash.com/blog/what-is-bootstrap-framework#3-admin-dashboards

DataSunrise. (s.f.). Integridad de datos en MySQL. https://www.datasunrise.com/es/centro-de-conocimiento/integridad-de-datos-en-mysql/

EBAC. (s.f.). Qué es GitHub. https://ebac.mx/blog/que-es-github

Fundació Jesuïtes Educació. (s.f.). ¿Qué es Bootstrap y cuáles son sus características? FP UOC. https://fp.uoc.fje.edu/es/estudios/blog-fp/que-es-bootstrap-y-cuales-son-sus-caracteristicas

GeeksforGeeks. (s.f.). PHP: Advantages and disadvantages of PHP. https://www.geeksforgeeks.org/php/advantages-and-disadvantages-of-php/

Microsoft. (s.f.). ¿Qué es Git? Learn Microsoft. https://learn.microsoft.com/es-es/devops/develop/git/what-is-git

Oracle. (s.f.). ¿Qué es MySQL? https://www.oracle.com/latam/mysql/what-is-mysql/

PHP Documentation Group. (s.f.). Introduction to PHP. PHP Manual. https://www.php.net/manual/es/introduction.php

The Bootstrap Authors. (s.f.). Getting started: Introduction. Bootstrap v4.1 Docs. https://getbootstrap.com/docs/4.1/getting-started/introduction

Contraslash. (2018, 20 de febrero). Mejores prácticas para el manejo de ramas en Git. GitHub. https://github.com/contraslash/blog_legacy/blob/master/posts_md/154-mejores-practicas-para-el-manejo-de-ramas-en-git.md

DreamHost. (2023, 30 de agosto). ¿PHP está muerto?. DreamHost Blog. https://www.dreamhost.com/blog/es/php-esta-muerto/

Gentoo Wiki. (s.f.). PHP. Recuperado el 7 de junio, de 2026, de https://wiki.gentoo.org/wiki/PHP

Mahendra, B. (2023, 28 de agosto). Using PHP with MySQL: Introduction and connection to database. Medium. https://medium.com/@bramahendramahendra1/using-php-with-mysql-introduction-and-connection-to-database-f69e62d3d57

MariaDB. (s.f.). MariaDB vs MySQL compatibility. MariaDB Knowledge Base. https://mariadb.com/docs/release-notes/community-server/about/compatibility-and-differences/mariadb-vs-mysql-compatibility

Piensa Solutions. (2023, 23 de noviembre). Qué es MySQL y cuáles son sus características. Piensa Solutions Blog. https://www.piensasolutions.com/blog/que-es-mysql-y-cuales-son-sus-caracteristicas#tree-2

W3Schools. (s.f.). PHP MySQL Connect. W3Schools. https://www.w3schools.com/php/php_mysql_connect.asp

Zend. (s.f.). Installing PHP on Linux. Zend Blog. https://www.zend.com/blog/installing-php-linux